第 5 部分（共 8 部分）· 解答

随机微积分，一步一步

第 5 部分：数学工具箱。
全部练习的完整解答



Rates with Bip

随机微积分，一步一步

第 5 部分：数学工具箱。练习解答

Rates with Bip

随机微积分，一步一步。第 5 部分：数学工具箱。练习解答。

© 2026 Rates with Bip。本书采用知识共享 署名-非商业性使用-禁止演绎 4.0 国际许可协议（CC BY-NC-ND 4.0）：你可以在注明作者 Rates with Bip 的前提下，为非商业目的原样复制和分享本书。许可协议：creativecommons.org/licenses/by-nc-nd/4.0。 配套笔记本以 MIT 许可协议免费提供：github.com/RatesWithBip/curriculum。

本书译自英文原版 Stochastic Calculus, Step by Step, Part 5: Solutions。代码、公式和数字与英文版完全相同。

全课程使用同一组数字：股价 $S_0 = 100$，行权价 $K = 105$，无风险利率 $r = 3\%$（连续复利），波动率 $\sigma = 20\%$，期限 $T = 1$ 年。第 5 部分还用到第 27 课的第二只股票（$S_0 = 95$，$\sigma = 30\%$，相关系数 0.5）、第 28 课的 Vasicek 利率（$\kappa = 0.5$，$\bar{r} = 4\%$，$s = 1\%$，$r_0 = 3\%$），以及几条虚构的小曲线和报价。这些都是为教学而虚构的，所有随机路径都用固定种子模拟。书中没有任何市场数据。

本册收录《随机微积分，一步一步》第 5 部分全部练习和自测题的完整解答。请先自己做每道练习，再看解答。

解答中的每个数字都由一个用断言自我检验的脚本计算；代码解答完整列出。

本书仅供学习之用，不构成任何金融、投资、法律或税务建议。

目录

如何使用这本解答 5

31. 多种风险的线性代数 6

32. 常微分方程 12

33. 复数与傅里叶变换 18

34. 数值方法：求根、插值、积分、拟合 24

35. 回归与偏微分方程进阶 31

如何使用这本解答

本册收录《随机微积分，一步一步》第 5 部分《数学工具箱》的完整解答：每道练习都完整解出，每一课末尾的自测题也给出答案。它应该在你自己尝试过每道题**之后**再读。

- **手算练习**（每课的前六道）完整解出。每个解答都从**思路**（打开局面的那一个想法）开始，然后分步骤展开，每一步都说明理由，最后以验证或解释结束。如果你卡住了，先只读思路，再自己试一试，然后才去读步骤。
- **编程练习**（每课的最后两道）给出完整的 Python 解答及其确切的输出。本册中的每个数字都由一个用断言自我检验的脚本生成，使用固定的随机种子，所以只要你用同样的种子，自己运行的结果应与书中显示的位数一致。
- **自测答案**很简短：一两句话，有数字的地方给出数字。

课程的数字始终不变：$S_0 = 100$，$K = 105$，$r = 3\%$（连续复利），$\sigma = 20\%$，$T = 1$ 年，看涨期权价值 7.1281。某一课需要时，还会用到第 27 课的第二只股票（$S_0 = 95$，$\sigma = 30\%$，相关系数 0.5）和第 28 课的 Vasicek 利率（$\kappa = 0.5$，$\bar{r} = 4\%$，$s = 1\%$，$r_0 = 3\%$）。

第 5 部分中几乎每个手算解答都用到三招之一：**展开再合并**，无论是 $LL^{\top}$ 的各个元素、泰勒级数的各项，还是债券方程中 r 的各次幂；**乘以合适的东西**，积分因子、共轭或检验函数，使剩下的只是单个导数或一个实数；或者**与已知情形核对**，正态分布、常系数或一条直线。如果你的答案不同，先检查常见的嫌疑：漏了转置，积分因子的符号，把 e^{-iuk} 写成 e^{iuk}，以及远期利率中忘了乘的那个 t。

本册仅供学习之用，不构成金融建议。

第 31 课解答

多种风险的线性代数

练习 31.1 至 31.8 以及第 31 课自测题的解答。

练习 31.1

思路。 展开和的方差（第 3 课），然后对一个权重最小化一个二次式。

第 1 步：各占一半。 取 $w = (0.5, 0.5)$：

$$\begin{aligned}\text{Var} &= 0.5^2(0.2)^2 + 0.5^2(0.3)^2 + 2(0.5)(0.5)(0.5)(0.2)(0.3) \\ &= 0.01 + 0.0225 + 0.015 = 0.0475,\end{aligned}$$

即波动率为 0.2179。

第 2 步：任意组合。 第一只股票占 w，第二只占 $1-w$，且 $\text{Cov} = \rho\sigma_1\sigma_2 = 0.03$：

$$V(w) = w^2\sigma_1^2 + (1-w)^2\sigma_2^2 + 2w(1-w)\rho\sigma_1\sigma_2.$$

第 3 步：令导数为零。

$$\begin{aligned}V'(w) &= 2w\sigma_1^2 - 2(1-w)\sigma_2^2 + 2(1-2w)\rho\sigma_1\sigma_2 = 0 \\ \Longrightarrow \quad w &= \frac{\sigma_2^2 - \rho\sigma_1\sigma_2}{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}.\end{aligned}$$

第 4 步：代入数字。 $w = (0.09 - 0.03)/(0.04 + 0.09 - 0.06) = 0.06/0.07 = 0.8571$，第二只股票占 0.1429。这时波动率为 0.1964：低于 20%，即单独持有较安全那只股票的波动率。加入一点风险更大的股票反而降低了风险，因为两只股票并不完全同步变动。

练习 31.2

思路。 把每个 X_i 写成各个 Z 的和，并利用 $k = l$ 时 $E[Z_k Z_l] = 1$，否则为 0。

协方差。 $X_i = \sum_k L_{ik} Z_k$ 的均值为 0，所以

$$\operatorname{Cov}(X_i,X_j)=E[X_iX_j]=\sum_k\sum_l L_{ik}L_{jl}E[Z_kZ_l]=\sum_k L_{ik}L_{jk}=(LL^{\top})_{ij}.$$

因子。 取 $L=\begin{pmatrix}a&0\\ b&c\end{pmatrix}$，则 $LL^{\top}=\begin{pmatrix}a^2&ab\\ ab&b^2+c^2\end{pmatrix}$。与 Σ 对应：

- $a^2=0.04$：$a=0.20$；
- $ab=0.03$：$b=0.15$；
- $b^2+c^2=0.09$：$c^2=0.09-0.0225=0.0675$，$c=0.2598$。

验证。 $LL^{\top}=\begin{pmatrix}0.04&0.2\times0.15\\ 0.15\times0.2&0.0225+0.0675\end{pmatrix}=\begin{pmatrix}0.04&0.03\\ 0.03&0.09\end{pmatrix}$。$\surd$

练习 31.3

思路。 一个方差为负的组合就足以排除一张表。要求出完整的范围，用行列式。

组合。 $w^{\top}Cw=\sum_i w_i^2+2\sum_{i<j}w_iw_j\rho_{ij}$。取 $w=(1,-1,1)$：$w_1w_2=-1$，$w_1w_3=1$，$w_2w_3=-1$，所以

$$w^{\top}Cw=3+2\big(-0.9+(-0.9)-0.9\big)=3-5.4=-2.4<0.$$

范围。 写 $c=\rho_{13}$，$a=0.9$。当 3×3 相关矩阵的行列式 ≥0 时它是有效的（它的 2×2 角块 $1-a^2$ 和 $1-c^2$ 在这里已经为正）。展开，

$$\det\begin{pmatrix}1&a&c\\ a&1&a\\ c&a&1\end{pmatrix}=1+2a^2c-2a^2-c^2.$$

由 $a^2=0.81$：$1+1.62c-1.62-c^2\ge0$，即 $c^2-1.62c+0.62\le0$，或 $(c-0.62)(c-1)\le0$。所以这张表恰好在 $\mathbf{0.62\le\rho_{13}\le1}$ 时可能成立。如果 1 与 2 的相关系数为 0.9，2 与 3 的也为 0.9，那么 1 与 3 的相关系数至少是 0.62。在 $c=-0.9$ 时，行列式为 -2.888。

练习 31.4

思路。 写 $C=(1-\rho)I+\rho\,\mathbf{1}\mathbf{1}^{\top}$，其中 $\mathbf{1}$ 是全 1 向量。

全 1 向量。 $\mathbf{1}^{\top}\mathbf{1}=n$，所以 $C\mathbf{1}=(1-\rho)\mathbf{1}+\rho\,\mathbf{1}\,n=\big(1+(n-1)\rho\big)\mathbf{1}$。特征值为 $1+(n-1)\rho$。

分量之和为零的向量。 若 $\mathbf{1}^\top v = 0$，则 $Cv = (1-\rho)v + \rho\,\mathbf{1}\,(\mathbf{1}^\top v) = (1-\rho)v$。特征值为 $1-\rho$。这些向量填满了其余 $n-1$ 个方向。

最小的公共相关系数。 所有特征值都必须 ≥ 0。对 $\rho \leq 1$，$1-\rho \geq 0$ 总是成立；起约束作用的条件是 $1+(n-1)\rho \geq 0$，所以 $\rho \geq -1/(n-1)$。五只股票两两之间的相关系数不可能都比 -0.25 更负：如果每只都在其他股票上涨时下跌，它们不可能同时都这样。取 $\rho = 0.5$，特征值为 3 和 0.5。

练习 31.5

思路。 逐列进行：对角元由剩余的方差得到，其下方的元素由剩余的协方差得到。

第 1 列。 $L_{11} = \sqrt{1} = 1$；$L_{21} = 0.5/1 = 0.5$；$L_{31} = 0.3/1 = 0.3$。

第 2 列。 $L_{22} = \sqrt{1 - 0.5^2} = \sqrt{0.75} = 0.8660$。$L_{32} = (\rho_{23} - L_{31}L_{21})/L_{22} = (0.4 - 0.15)/0.8660 = 0.2887$。

第 3 列。 $L_{33} = \sqrt{1 - 0.3^2 - 0.2887^2} = \sqrt{1 - 0.09 - 0.0833} = 0.9092$。

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 0.8660 & 0 \\ 0.3 & 0.2887 & 0.9092 \end{pmatrix}.$$

解读。 股票 3 取了股票 1 的噪声的 0.3、股票 2 中新噪声的 0.2887，以及它自己的噪声的 0.9092：它们的平方加起来等于 1，正如相关矩阵的对角元所要求的。

练习 31.6

思路。 每个权重都是 $1/n$ 时的 $w^\top \Sigma w$：n 个对角项和 $n(n-1)$ 个非对角项。

第 1 步。

$$\text{Var} = \frac{1}{n^2}\Big(n\sigma^2 + n(n-1)\rho\sigma^2\Big) = \sigma^2\,\frac{1+(n-1)\rho}{n}.$$

第 2 步：代入数字。 $n = 5$，$\rho = 0.5$：$\sigma\sqrt{3/5} = 0.1549$。

第 3 步：极限。 $\frac{1+(n-1)\rho}{n} = \frac{1}{n} + \rho\frac{n-1}{n} \to \rho$，所以波动率趋于 $\sigma\sqrt{\rho} = 0.1414$。

含义。 分散化消除了每只股票风险中它自己独有的部分，但消除不了所有股票共有的部分。相关系数为 0.5 时，由这些股票组成的篮子无论多大，波动率都不会低于 14.1%。

练习 31.7

带主元检查的 Cholesky 分解，与 numpy 对照检验。

```
import numpy as np


def cholesky(A):
    # Returns L with L @ L.T = A, or raises with the failing pivot.
    n = len(A)
    L = np.zeros((n, n))
    for j in range(n):
        d = A[j, j] - L[j, :j] @ L[j, :j]
        if d <= 0:
            raise ValueError(f"pivot {j + 1} is {d:.4f}: not positive definite")
        L[j, j] = math.sqrt(d)
        L[j + 1:, j] = (A[j + 1:, j] - L[j + 1:, :j] @ L[j, :j]) / L[j, j]
    return L


tests = {"two shares": np.array([[0.04, 0.03], [0.03, 0.09]]),
         "three shares": np.array([[1.0, 0.5, 0.3], [0.5, 1.0, 0.4], [0.3, 0.4, 1.0]]),
         "impossible table": np.array([[1.0, 0.9, -0.9], [0.9, 1.0, 0.9], [-0.9, 0.9, 1.0]])}
for name, A in tests.items():
    try:
        L = cholesky(A)
        print(f"{name:17s} ok, largest |LL' - A| = {np.abs(L @ L.T - A).max():.1e}")
    except ValueError as e:
        print(f"{name:17s} {e}")

rng = np.random.default_rng(317)
worst = 0.0
for _ in range(1000):
    G = rng.standard_normal((6, 12))
    C = np.corrcoef(G)                      # a random valid correlation matrix
    worst = max(worst, np.abs(cholesky(C) - np.linalg.cholesky(C)).max())
print(f"1000 random 6 x 6 correlation matrices: largest gap from numpy {worst:.1e}")
```

它的输出。

```
two shares        ok, largest |LL' - A| = 1.4e-17
three shares      ok, largest |LL' - A| = 1.1e-16
impossible table  pivot 3 is -15.2000: not positive definite
1000 random 6 x 6 correlation matrices: largest gap from numpy 7.2e-16
```

读输出。 在 1000 个随机矩阵上，亲手写的程序与 numpy 的结果在舍入误差范围内一致。在不可能的那张表上，它停在第三个主元处，那里剩余的方差 $1 - L_{31}^2 - L_{32}^2$ 为负：股票 1 和 2 用掉的比股票 3 的全部方差还多。

练习 31.8

篮子看涨期权，与按篮子波动率计算的 Black-Scholes 看涨期权对比。

```
def bs_call(s, k, t, vol):
    d1 = (math.log(s / k) + (r + 0.5 * vol**2) * t) / (vol * math.sqrt(t))
    return s * N(d1) - k * math.exp(-r * t) * N(d1 - vol * math.sqrt(t))


rng = np.random.default_rng(318)
Z = rng.standard_normal((400_000, 5))
print("  rho   basket (MC)     se    vol    BS at that vol    gap")
res = {}
for rho_ in (0.0, 0.25, 0.5, 0.75, 1.0):
    C = np.full((5, 5), rho_) + (1 - rho_) * np.eye(5)
    if rho_ < 1:
        X = Z @ cholesky(C).T
    else:
        X = np.repeat(Z[:, :1], 5, axis=1)  # perfectly correlated: one draw for all
    S = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * X)
    pay = math.exp(-r * T) * np.maximum(S.mean(axis=1) - K, 0)
    vol = sigma * math.sqrt((1 + 4 * rho_) / 5)
    approx = bs_call(S0, K, T, vol)
    res[rho_] = (pay.mean(), approx)
    print(f"{rho_:5.2f}  {pay.mean():9.4f}  {pay.std() / math.sqrt(len(pay)):7.4f}  {vol:.4f}
{approx:9.4f}   {pay.mean() - approx:+.4f}")
```

它的输出。

```
  rho   basket (MC)     se    vol    BS at that vol    gap
 0.00     2.7644   0.0078  0.0894      2.7316   +0.0328
 0.25     4.2134   0.0116  0.1265      4.1983   +0.0151
 0.50     5.3350   0.0147  0.1549      5.3301   +0.0049
 0.75     6.2832   0.0174  0.1789      6.2857   -0.0025
 1.00     7.1120   0.0198  0.2000      7.1281   -0.0161
```

读输出。 近似在 $\rho = 0$ 时最差，差了 0.0328，在 0.5 到 0.75 之间最好。独立的对数正态价格的平均不是对数正态的：它的分布偏斜较小，而波动率相同的对数正态分布给高结果的权重太小。随着 ρ 上升，五只股票越来越像一只股票那样变动；到 $\rho = 1$ 时，篮子*就是*一只波动率为 20% 的股票，所以 Black-Scholes 价格是精确的：-0.0161 的差距在模拟的一个标准误差之内。

自测

1. 因为 $w^\top \Sigma w$ 是组合 $w^\top R$ 的方差，而方差不可能为负。

2. $L = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{pmatrix}$：所以 $X_2 = \rho Z_1 + \sqrt{1-\rho^2}\, Z_2$。

3. 3，特征向量为 $(1,1,1,1,1)/\sqrt{5}$：五只股票一起变动，即“市场”。它承担了总方差的 0.60。

第 32 课解答

常微分方程

练习 32.1 至 32.8 以及第 32 课自测题的解答。

练习 32.1

思路。 把 m 项移到左边，再乘以 $e^{\kappa t}$，使左边成为单个导数。

第 1 步。 $m' + \kappa m = \kappa \bar{r}$。乘以 $e^{\kappa t}$：$(e^{\kappa t} m)' = \kappa \bar{r} e^{\kappa t}$。

第 2 步：从 0 积分到 t。 $e^{\kappa t} m(t) - r_0 = \bar{r}(e^{\kappa t} - 1)$。

第 3 步：相除。 $m(t) = \bar{r} + (r_0 - \bar{r}) e^{-\kappa t}$。

代入数字。 $e^{-0.5} = 0.6065$，$e^{-2.5} = 0.0821$，所以 $E[r_1] = 4\% - 1\% \times 0.6065 = 0.03393$，$E[r_5] = 0.03918$。

一半的差距。 当 $t = \ln 2 / \kappa = 1.386$ 年时，$e^{-\kappa t} = \tfrac{1}{2}$。此后每再过 1.39 年，剩下的差距就再缩小一半。

练习 32.2

思路。 同一个方程，取 $a = 2\kappa$，右边是常数 s^2。

第 1 步。 $(e^{2\kappa t} v)' = s^2 e^{2\kappa t}$，所以 $e^{2\kappa t} v(t) = \frac{s^2}{2\kappa}(e^{2\kappa t} - 1)$，且

$$v(t) = \frac{s^2}{2\kappa}\left(1 - e^{-2\kappa t}\right).$$

第 2 步：长期。 $v \to s^2/(2\kappa)$，标准差为 $s/\sqrt{2\kappa} = 0.01/1 = 0.0100$。

第 3 步：一年时。 $v(1) = 0.0001 \times (1 - e^{-1}) = 6.3\text{e-}05$，标准差为 0.00795。

含义。 如果没有向 $\bar{r}$ 的回拉（$\kappa = 0$），方差将是 $s^2 t$，永远增长。有了回拉，新的随机性以速率 s^2 加入，旧的随机性以速率 2κ 衰减，两者达到平衡。

练习 32.3

思路。把仿射猜测代入第 28 课的偏微分方程，把含 r 的项与其余的项分开。

第 1 步。偏微分方程为 $P_t+\kappa(\bar r-r)P_r+\tfrac12 s^2P_{rr}=rP$。取 $P=e^{A(\tau)-B(\tau)r}$，$\tau=T-t$：$P_t=-(A'-B'r)P$，$P_r=-BP$，$P_{rr}=B^2P$。

第 2 步：除以 P。$-A'+B'r-\kappa(\bar r-r)B+\tfrac12 s^2B^2=r$。

第 3 步：合并。含 r 的项：$B'+\kappa B=1$。不含 r 的项：$-A'-\kappa\bar r B+\tfrac12 s^2B^2=0$，即 $A'=-\kappa\bar r B+\tfrac12 s^2B^2$。到期时无论 r 是多少，债券都支付 1，所以 $A(0)=B(0)=0$。

第 4 步：解出 B。$(e^{\kappa\tau}B)'=e^{\kappa\tau}$，所以 $e^{\kappa\tau}B=(e^{\kappa\tau}-1)/\kappa$，$B(\tau)=(1-e^{-\kappa\tau})/\kappa$。当 $\tau\to\infty$ 时，$B\to 1/\kappa=2$；在 10 年时它已经是 1.9865。

为什么 s^2 进入 A。在 Vasicek 中，波动率 s 不依赖于 r，所以 $\tfrac12 s^2B^2$ 中没有 r，落在 A 的方程里。在 CIR 中它乘以 r，于是落在 B 的方程里（练习 32.4）。

练习 32.4

思路。对代换式求导，看平方项如何相消。

第 1 步。取 $y=-u'/(cu)$：

$$y'=-\frac{u''}{cu}+\frac{(u')^2}{cu^2},\qquad cy^2=\frac{(u')^2}{cu^2}.$$

第 2 步：代入 $y'=a+by+cy^2$。$(u')^2/(cu^2)$ 项在两边都出现并相消，剩下 $-\dfrac{u''}{cu}=a-\dfrac{bu'}{cu}$。乘以 $-cu$：

$$u''-bu'+acu=0.$$

第 3 步：CIR 的情形。$a=1$，$b=-\kappa$，$c=-\tfrac12 s_c^2$（其中 $s_c^2=0.003333$），所以 $u''+\kappa u'-\tfrac12 s_c^2u=0$。试探 $u=e^{\lambda\tau}$：

$$\lambda^2+\kappa\lambda-\tfrac12 s_c^2=0,\qquad \lambda=\tfrac12\big(-\kappa\pm\gamma\big),\quad \gamma=\sqrt{\kappa^2+2s_c^2}=0.506623.$$

两个指数分别为 0.003311 和 -0.503311。

第 4 步：长期。 τ 很大时，指数较大的那一项在 u 中占主导，所以 $u'/u \to \lambda_+$，$B = 2u'/(s_c^2 u) \to 2\lambda_+/s_c^2 = (\gamma - \kappa)/s_c^2 = 1.9868$。同一个数也是 $B' = 0$ 之处：$1 - \kappa B - \tfrac{1}{2} s_c^2 B^2 = 0$ 的正根。

练习 32.5

思路。 分离变量：含 y 的都放在一边，dt 放在另一边。

$y' = y^2$，$y(0) = 1$。 $dy/y^2 = dt$，所以 $-1/y = t + C$；在 $t = 0$ 处，$C = -1$。所以 $y = 1/(1 - t)$，在 $t = 1$ 时爆炸。

$y' = 1 + y^2$，$y(0) = 0$。 $dy/(1 + y^2) = dt$，所以 $\arctan y = t$，$y = \tan t$，在 $t = \pi/2 = 1.5708$ 时爆炸。

$y' = 1 - y^2$，$y(0) = 0$。 $dy/(1 - y^2) = dt$，所以 $\tfrac{1}{2}\ln\tfrac{1+y}{1-y} = t$，$y = \tanh t$，它升向 1，但永远达不到。

为什么符号起决定作用。 取 $+y^2$ 时，y 越大，增长越快：增长以比指数更快的速度自我强化，在有限时刻 t^* 附近 y 的行为像 $1/(t^* - t)$。取 $-y^2$ 时，平方项起刹车作用：在 $y = 1$ 处 $y' = 0$，这是解趋近但无法越过的水平。

练习 32.6

思路。 每个欧拉步都乘以同一个因子。

第 1 步。 $y_{n+1} = y_n + h(-\kappa y_n) = (1 - \kappa h) y_n$，所以 $y_n = (1 - \kappa h)^n$。

第 2 步：代入数字。 $0.95^{10} = 0.598737$，而 $e^{-0.5} = 0.606531$：误差为 0.00779。步长减半：$0.975^{20} = 0.602688$，误差为 0.00384。两个误差之比为 2.03。

第 3 步：为什么误差与 h 成正比。 取 $n = t/h$，$\ln y_n = \tfrac{t}{h}\ln(1 - \kappa h) = \tfrac{t}{h}\big(-\kappa h - \tfrac{1}{2}\kappa^2 h^2 - \cdots\big) = -\kappa t - \tfrac{1}{2}\kappa^2 t h - \cdots$。所以 $y_n \approx e^{-\kappa t}\big(1 - \tfrac{1}{2}\kappa^2 t h\big)$：误差为 $\tfrac{1}{2}\kappa^2 t\, h\, e^{-\kappa t}$，与 h 成正比。取 $t = 1$，$h = 0.1$：$0.0125 \times 0.6065 = 0.0076$，与实测的 0.0078 很接近。

第 4 步：步长太大。 若 $h > 2/\kappa = 4$，则 $|1 - \kappa h| > 1$：欧拉法的值不断增大，每步变号一次，而真实解在衰减。这与第 29 课显式格式的不稳定性是同一回事。

练习 32.7

用 Runge-Kutta 4 计算 CIR 收益率曲线，与闭式解对比。

```
import numpy as np


def closed_form(tau):
    e = math.exp(gam * tau) - 1
    B = 2 * e / ((gam + kappa) * e + 2 * gam)
    A = (2 * kappa * rbar / sc**2) * math.log(2 * gam * math.exp((gam + kappa) * tau / 2) /
((gam + kappa) * e + 2 * gam))
    return A, B


def f(y):
    return np.array([-kappa * rbar * y[1], 1 - kappa * y[1] - 0.5 * sc**2 * y[1] ** 2])


h = 0.1
y = np.zeros(2)
print(" years   RK4 yield %   closed form %   gap (bp)")
worst = 0.0
for n in range(1, 301):
    k1 = f(y); k2 = f(y + h / 2 * k1); k3 = f(y + h / 2 * k2); k4 = f(y + h * k3)
    y = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    tau = round(n * h, 10)
    if abs(tau - round(tau)) < 1e-9:
        A, B = closed_form(tau)
        y_rk = -(y[0] - y[1] * r0) / tau * 100
        y_cf = -(A - B * r0) / tau * 100
        worst = max(worst, abs(y_rk - y_cf) * 100)
        if round(tau) in (1, 2, 5, 10, 20, 30):
            print(f"{tau:6.0f}   {y_rk:10.6f}    {y_cf:10.6f}    {(y_rk - y_cf) * 100:+.2e}")
print(f"largest gap over 1 to 30 years: {worst:.1e} bp")
```

它的输出。

```
 years   RK4 yield %   closed form %   gap (bp)
     1     3.211850      3.211850    +3.03e-06
     2     3.364262      3.364262    +1.81e-06
     5     3.622063      3.622063    +3.61e-07
    10     3.784009      3.784009    -1.43e-09
    20     3.878252      3.878252    -1.77e-08
    30     3.910060      3.910060    -1.20e-08
largest gap over 1 to 30 years: 3.0e-06 bp
```

读输出。 用 300 个 0.1 年的步长，1 至 30 年上的最大差距为 3.0e-06 个基点：远低于任何要紧的水平。对于没有闭式解的曲线，在 Riccati 方程组上用 Runge-Kutta 和公式一样好。

练习 32.8

在 $y' = 1 + y^2$ 上做步长加倍。

```
g = lambda y: 1 + y * y


def rk4_step(y, h):
    k1 = g(y); k2 = g(y + h / 2 * k1); k3 = g(y + h / 2 * k2); k4 = g(y + h * k3)
    return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)


h = 0.01
y, t, first = 0.0, 0.0, None
print("    t        y      tan t    estimate")
while t < 1.57 - 1e-9:
    one = rk4_step(y, h)
    two = rk4_step(rk4_step(y, h / 2), h / 2)
    est = abs(one - two)
    y, t = two, round(t + h, 10)
    if first is None and est > 1e-6:
        first = t
    if abs(t * 100 - round(t * 100)) < 1e-9 and (round(t * 100) % 10 == 0 or t > 1.5):
        print(f"{t:5.2f}  {y:9.3f}  {math.tan(t):9.3f}  {est:.1e}")
print(f"the estimate first exceeds 1e-6 at t = {first:.2f}; the blow-up is at pi/2 = {math.pi
/ 2:.4f}")
```

它的输出。

```
    t        y      tan t    estimate
 0.10      0.100      0.100  7.2e-13
 0.20      0.203      0.203  5.1e-13
 0.30      0.309      0.309  1.2e-13
 0.40      0.423      0.423  5.1e-13
 0.50      0.546      0.546  1.4e-12
 0.60      0.684      0.684  2.8e-12
 0.70      0.842      0.842  4.4e-12
 0.80      1.030      1.030  5.9e-12
 0.90      1.260      1.260  3.7e-12
 1.00      1.557      1.557  1.8e-11
 1.10      1.965      1.965  1.4e-10
 1.20      2.572      2.572  8.8e-10
 1.30      3.602      3.602  7.3e-09
 1.40      5.798      5.798  1.3e-07
 1.50     14.101     14.101  2.3e-05
 1.51     16.428     16.428  5.4e-05
 1.52     19.670     19.670  1.5e-04
 1.53     24.498     24.498  5.0e-04
 1.54     32.461     32.461  2.3e-03
 1.55     48.076     48.078  1.7e-02
 1.56     92.573     92.620  3.7e-01
 1.57    868.654   1255.766  2.8e+02
the estimate first exceeds 1e-6 at t = 1.45; the blow-up is at pi/2 = 1.5708
```

读输出。估计在 $t = 1.45$ 处首次超过 10^{-6}，在爆炸之前约 0.12，而这时解本身仍精确到六位。此后估计大约每 0.02 增大 10 倍。一个在估计过大时就把步长减半的求解器会走越来越小的步，在 $\pi/2$ 之前停住，这正是我们想要的警告。

自测

1. e^{3t}。于是 $(e^{3t}y)' = e^{3t}$，所以 $y = \tfrac{1}{3}(1 - e^{-3t})$。

2. 在 CIR 中波动率为 $s_c\sqrt{r}$，所以偏微分方程中的项 $\tfrac{1}{2}s_c^2 r\, P_{rr} = \tfrac{1}{2}s_c^2 B^2\, r\, P$ 含有 r，落在 B 的方程（r 的系数）中。在 Vasicek 中，$\tfrac{1}{2}s^2B^2$ 不含 r，落在 A 的方程中。

3. 大约缩小到十六分之一（在本课中为 1/16.9）：Runge-Kutta 4 的误差与 h^4 成正比。

第 33 课解答

复数与傅里叶变换

练习 33.1 至 33.8 以及第 33 课自测题的解答。

练习 33.1

思路。 乘开，把 i^2 换成 -1；做除法时，分子分母同乘以共轭。

乘积。 $(3+4i)(1-2i) = 3 - 6i + 4i - 8i^2 = 3 - 2i + 8 = 11 - 2i$。

模。 $|3+4i| = \sqrt{9+16} = 5$。

倒数。 $\dfrac{1}{3+4i} = \dfrac{3-4i}{(3+4i)(3-4i)} = \dfrac{3-4i}{25} = 0.12 - 0.16\mathrm{i}$。

乘以 i。 $i(a+bi) = ai + bi^2 = -b + ai$：点 (a,b) 移到 $(-b,a)$。它的长度 $\sqrt{b^2+a^2}$ 不变。而且 $(a,b)\cdot(-b,a) = -ab + ab = 0$：两者互相垂直。$(1,0)$ 变到 $(0,1)$，说明转动是逆时针的。

练习 33.2

思路。 按 i 的幂把 $e^{i\theta}$ 的级数拆开，i 的幂依次循环为 $1, i, -1, -i$。

欧拉公式。 在 $e^{i\theta} = \sum_n (i\theta)^n/n!$ 中，偶次幂给出 $(-1)^k\theta^{2k}/(2k)!$，这是 $\cos\theta$ 的级数；奇次幂给出 $i(-1)^k\theta^{2k+1}/(2k+1)!$，这是 i 乘以 $\sin\theta$ 的级数。

模。 $e^{i\theta}$ 的共轭是 $\cos\theta - i\sin\theta = e^{-i\theta}$，所以 $|e^{i\theta}|^2 = e^{i\theta}e^{-i\theta} = e^0 = 1$。（等价地，$\cos^2\theta + \sin^2\theta = 1$。）

和角公式。 把 $e^{ia}e^{ib} = (\cos a + i\sin a)(\cos b + i\sin b)$ 乘开：

$$= (\cos a\cos b - \sin a\sin b) + i(\sin a\cos b + \cos a\sin b).$$

它等于 $e^{i(a+b)} = \cos(a+b) + i\sin(a+b)$。让实部和虚部分别相等，得到 $\cos(a+b) = \cos a\cos b - \sin a\sin b$ 和 $\sin(a+b) = \sin a\cos b + \cos a\sin b$。

练习 33.3

思路。 在指数中配方，和求矩母函数时完全一样。

第 1 步。

$$E[e^{iuX}] = \int \frac{1}{\sqrt{2\pi s^2}} \exp\Big(-\frac{(x-m)^2}{2s^2} + iux \Big) dx.$$

第 2 步：配方。

$$-\frac{(x-m)^2}{2s^2} + iux = -\frac{(x - m - ius^2)^2}{2s^2} + ium - \tfrac{1}{2}s^2u^2.$$

（展开右边的平方来验证：交叉项给出 $iux - ium$，而 $-(ius^2)^2/(2s^2) = \tfrac{1}{2}s^2u^2$。）

第 3 步。 因子 $e^{ium - \frac{1}{2}s^2u^2}$ 提到积分外面。剩下的是平移了 ius^2 的正态密度，它的积分为 1，和实数平移时一样。（把积分路径平移一个虚数量需要复分析中的一个简短论证；答案也可以通过级数 $\sum (iu)^n E[X^n]/n!$ 来验证。）所以 $\varphi(u) = e^{ium - \frac{1}{2}s^2u^2}$。

对 $\ln S_T$。 $m = \ln 100 + (0.03 - 0.02) \times 1 = 4.60517 + 0.01 = 4.61517$，$s^2 = 0.04$：

$$\varphi(u) = \exp\big(4.61517\, iu - 0.02\, u^2 \big).$$

练习 33.4

思路。 e^{iuX} 位于单位圆上。

- $\varphi(0) = E[e^0] = 1$。
- $|\varphi(u)| = |E[e^{iuX}]| \le E[|e^{iuX}|] = E[1] = 1$：平均值的大小不超过大小的平均值。
- $\varphi(-u) = E[e^{-iuX}] = E\big[\overline{e^{iuX}}\big] = \overline{E[e^{iuX}]} = \overline{\varphi(u)}$。

S_T 没有矩母函数。 $E[e^{tS_T}] = E\big[\exp(te^X)\big]$，其中 X 服从正态分布。x 很大时，$\exp(te^x)$ 像指数的指数那样增长，而正态密度只像 $e^{-x^2/(2s^2)}$ 那样下降。乘积无限增长，所以对每个 $t > 0$ 积分都是无穷大。特征函数完全不需要这种条件：无论 X 是什么，$|e^{iuX}| = 1$。

练习 33.5

思路。 把权重 e^X 并入指数：$e^{iuX} e^X = e^{i(u-i)X}$。

第 1 步。 在股票测度下，$E^S[g(X)] = E[g(X)e^X]/E[e^X]$。所以

$$\varphi_S(u) = \frac{E[e^{iuX}e^X]}{E[e^X]} = \frac{E[e^{i(u-i)X}]}{E[e^{i(-i)X}]} = \frac{\varphi(u-i)}{\varphi(-i)},$$

因为 $i(u-i) = iu + 1$，$i(-i) = 1$。

第 2 步：Black-Scholes 的情形。 $\varphi(u-i) = \exp\left(i(u-i)m - \frac{1}{2}v(u-i)^2\right) = \exp\left(ium + m - \frac{1}{2}vu^2 + ivu + \frac{1}{2}v\right)$，而 $\varphi(-i) = \exp(m + \frac{1}{2}v)$。相除：

$$\varphi_S(u) = \exp\left(iu(m+v) - \tfrac{1}{2}vu^2\right),$$

这是方差相同、均值为 $m + v = \ln S_0 + (r + \frac{1}{2}\sigma^2)T = 4.65517$ 的正态分布的特征函数。这就是第 27 课的结果：在股票测度下，对数价格的漂移为 $r + \frac{1}{2}\sigma^2$，这里为 0.05。而 $\varphi(-i) = E[S_T] = 103.0455$。

练习 33.6

思路。 把两个因子都展开到 u 的一阶。

第 1 步。 $\varphi(u) = E[1 + iuX + O(u^2)] = 1 + iuE[X] + O(u^2)$，$e^{-iuk} = 1 - iuk + O(u^2)$。

第 2 步。 它们的乘积是 $1 + iu(E[X] - k) + O(u^2)$。除以 iu：

$$\frac{e^{-iuk}\varphi(u)}{iu} = \frac{1}{iu} + (E[X] - k) + O(u).$$

第 3 步。 $1/(iu) = -i/u$ 是纯虚数，所以它的实部为零。整体的实部趋于 $E[X] - k$。

代入数字。 $E[\ln S_T] - \ln K = \ln(100/105) + 0.01 = -0.04879 + 0.01 = -0.03879$。为负，因为行权价高于平均对数价格。

练习 33.7

十一个行权价的 Gil-Pelaez 价格，以及它们所需的积分设置。

```
import numpy as np

v = sigma**2 * T
cf = lambda u: np.exp(1j * u * m - 0.5 * v * u**2)
cf_share = lambda u: cf(u - 1j) / cf(-1j)


def prob_above(phi, k, U, du):
    u = np.arange(du / 2, U, du)
    return 0.5 + (np.exp(-1j * u * k) * phi(u) / (1j * u)).real.sum() * du / math.pi


def fourier_call(k_strike, U, du):
    k = math.log(k_strike)
    return S0 * prob_above(cf_share, k, U, du) - k_strike * math.exp(-r * T) * prob_above(cf,
k, U, du)


def bs_call(k_strike):
    d1 = (math.log(S0 / k_strike) + (r + 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
    return S0 * N(d1) - k_strike * math.exp(-r * T) * N(d1 - sigma * math.sqrt(T))


strikes = range(80, 135, 5)
print("   U    du   worst error over strikes 80-130")
best = None
for U in (10, 15, 20, 25, 30):
    for du in (4.0, 0.5):
        worst = max(abs(fourier_call(k, U, du) - bs_call(k)) for k in strikes)
        print(f"{U:4d}  {du:4.1f}   {worst:.1e}")
        if best is None and worst < 5e-5:
            best = (U, du)
U, du = best
print(f"first (U, du) with four correct decimals at every strike: U = {U}, du = {du}")
print("strike   Fourier    formula")
for k in strikes:
    print(f"{k:6d}  {fourier_call(k, U, du):8.4f}  {bs_call(k):8.4f}")
```

它的输出。

```
   U    du   worst error over strikes 80-130
  10   4.0   7.9e-01
  10   0.5   3.6e-01
  15   4.0   8.9e-03
  15   0.5   2.3e-02
  20   4.0   3.4e-04
  20   0.5   5.2e-04
  25   4.0   6.9e-06
  25   0.5   4.4e-06
  30   4.0   7.8e-08
  30   0.5   1.5e-08
first (U, du) with four correct decimals at every strike: U = 25, du = 4.0
strike   Fourier    formula
    80   23.2240   23.2240
    85   19.1313   19.1313
    90   15.4292   15.4292
    95   12.1797   12.1797
   100    9.4134    9.4134
   105    7.1281    7.1281
   110    5.2934    5.2934
   115    3.8596    3.8596
   120    2.7666    2.7666
   125    1.9521    1.9521
   130    1.3577    1.3577
```

读输出。 要紧的是上限：$U = 25$ 在每个行权价上都给出四位正确的小数，即使用粗步长 $du = 4$ 也是如此。被积函数像 $e^{-\frac{1}{2}\sigma^2 T u^2} = e^{-0.02u^2}$ 那样衰减，在 $u = 25$ 处约为 4×10^{-6}。$\sigma\sqrt{T}$ 较小时（期限更短，或波动率更低），它衰减得更慢，U 必须按 $1/(\sigma\sqrt{T})$ 的比例增大。

练习 33.8

沿一条路径的复数平方根，两种算法。

```
u = np.linspace(0, 6 * math.pi, 1201)
zu = np.exp(1j * u) + 0.3
naive = np.exp(0.5 * np.log(zu))                          # principal log: angle in (-pi, pi]
smooth = np.sqrt(np.abs(zu)) * np.exp(0.5j * np.unwrap(np.angle(zu)))
jn = np.abs(np.diff(naive))
js = np.abs(np.diff(smooth))
print(f"principal log: {int((jn > 0.5).sum())} jumps, largest step {jn.max():.3f}")
print(f"unwrapped:     {int((js > 0.5).sum())} jumps, largest step {js.max():.4f}")
print("where the principal version jumps (u / pi):", np.round(u[1:][jn > 0.5] / math.pi,
3).tolist())
print(f"the two agree up to sign everywhere: max | |naive| - |smooth| | =
{np.abs(np.abs(naive) - np.abs(smooth)).max():.1e}")
```

它的输出。

```
principal log: 3 jumps, largest step 1.673
unwrapped:     0 jumps, largest step 0.0094
where the principal version jumps (u / pi): [1.0, 3.005, 5.0]
the two agree up to sign everywhere: max | |naive| - |smooth| | = 6.7e-16
```

读输出。 主值版本跳跃 3 次，在 $u = \pi$、3π 和 5π 处：恰好是 $z(u) = e^{iu} + 0.3$ 穿过负实轴的地方（在 $u = \pi$ 处，$z = -0.7$）。在那里它的角度从 π 翻到 $-\pi$，平方根随之变号。两个版本处处大小相同；它们只在交替的区段上符号不同。

在价格中。 对这样一个函数的积分会把符号错误的区段加进去。结果是一个错误的数，没有任何报错，甚至可能看起来很合理。跟踪角度（或者像第 48 课对 Heston 所做的那样改写公式，使复数永不穿过负实轴）就能避免它。

自测

1. 1：$e^{i\theta} = \cos\theta + i\sin\theta$，而 $\cos^2\theta + \sin^2\theta = 1$。它是单位圆上角度为 θ 的点。

2. $e^{-u^2/2}$。

3. 因为对这些模型，特征函数有公式，而密度没有。傅里叶反演把特征函数变成概率，从而得到价格。

第 34 课解答

数值方法：求根、插值、积分、拟合

练习 34.1 至 34.8 以及第 34 课自测题的解答。

练习 34.1

思路。 二分法每步把宽度减半；牛顿法每步把误差平方。

二分法。 减半 n 次后，宽度为 $0.99/2^n$。需要 $0.99/2^n \le 10^{-8}$，所以 $2^n \ge 0.99 \times 10^8$，即 $n \ge \log_2(0.99 \times 10^8) = 26.56$。所以需要减半 27 次。

牛顿法。 $10^{-2} \to 10^{-4} \to 10^{-8} \to 10^{-16}$：三步之后误差就低于 10^{-12}。正确的位数每步翻一番：2、4、8、16。

练习 34.2

思路。 把函数换成它的切线，求切线与零的交点。

更新公式。 由泰勒定理，$C(\sigma_n + \delta) \approx C(\sigma_n) + \mathcal{V}(\sigma_n)\,\delta$。令它等于 C^*，得到 $\delta = -\big(C(\sigma_n) - C^*\big)/\mathcal{V}(\sigma_n)$，于是 $\sigma_{n+1} = \sigma_n + \delta$。

为什么会失败。 这一步要除以 vega。在深度虚值、低波动率时，期权几乎没有机会产生收益，它的价格对 σ 几乎没有反应：vega 接近零（本课的例子：约 10^{-37}）。这时切线几乎是平的，它与零的交点非常远，下一个迭代值荒谬可笑。牛顿法的速度来自信任切线，而这里切线毫无用处。

Brent 法的做法。 它保留一个使价格误差变号的区间 $[a, b]$，所以根永远跑不掉。它尝试一个快速的插值步，只有当这一步留在区间内并使区间缩小得足够多时才接受；否则就二分。所以它永远不比二分法差，而且通常几乎和牛顿法一样快。

练习 34.3

思路。 在每个区间上，三次多项式的二阶导数是一条直线。积分两次，用两端的值确定常数，然后让节点处的斜率相等。

第 1 步：二阶导数。 在 $[t_i, t_{i+1}]$ 上，S'' 是线性的，从 M_i 变到 M_{i+1}：

$$S''(t) = M_i \frac{t_{i+1} - t}{h_i} + M_{i+1} \frac{t - t_i}{h_i}.$$

第 2 步：积分两次，并要求 $S(t_i) = y_i$、$S(t_{i+1}) = y_{i+1}$：

$$\begin{aligned} S(t) = {} & M_i \frac{(t_{i+1} - t)^3}{6h_i} + M_{i+1} \frac{(t - t_i)^3}{6h_i} \\ & + \left(\frac{y_i}{h_i} - \frac{M_i h_i}{6} \right)(t_{i+1} - t) + \left(\frac{y_{i+1}}{h_i} - \frac{M_{i+1} h_i}{6} \right)(t - t_i). \end{aligned}$$

第 3 步：两端的斜率。 求导并代入：

$$S'(t_i^+) = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{3} M_i - \frac{h_i}{6} M_{i+1}, \qquad S'(t_{i+1}^-) = \frac{y_{i+1} - y_i}{h_i} + \frac{h_i}{6} M_i + \frac{h_i}{3} M_{i+1}.$$

第 4 步：让 t_i 处的斜率相等。 从左边区间到达的斜率 $\frac{y_i - y_{i-1}}{h_{i-1}} + \frac{h_{i-1}}{6} M_{i-1} + \frac{h_{i-1}}{3} M_i$ 必须等于向右离开的斜率 $\frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{3} M_i - \frac{h_i}{6} M_{i+1}$。乘以 6 并合并各个 M：

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i) M_i + h_i M_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right).$$

值和二阶导数按构造是连续的；这个方程使斜率也连续。取 $M_0 = M_n = 0$（自然端点），它是一个三对角方程组。

练习 34.4

思路。 在各点之间，直线插值的零息利率斜率不变，斜率在每个点处改变。

2 年之前。 在 $[1, 2]$ 上，$z' = (3.3 - 3.0)/1 = 0.3\%$ 每年，所以 $f(2^-) = 3.3 + 2 \times 0.3 = 3.9\%$。

2 年之后。 在 $[2, 3]$ 上，$z' = 0.2\%$ 每年，所以 $f(2^+) = 3.3 + 2 \times 0.2 = 3.7\%$。

为什么这很重要。 远期利率在一个任意的日期下跌 0.2%，等于说市场预期利率恰好在那一天突然下降，没有人会相信这一点。依赖于 2 年之后利率的合约，其定价与依赖于 2 年之前利率的合约不同，而且随着报价变动，它们对报价利率的敏感度会四处跳动。任何建立在远期利率上的产品的价格都继承了这把锯子。

练习 34.5

思路。 逐个幂次直接验证。

$f(x)$	$\int_{-1}^{1} f\,dx$	$f(-1/\sqrt{3})+f(1/\sqrt{3})$
1	2	2
x	0	0
x^2	$2/3$	$1/3+1/3=2/3$
x^3	0	0
x^4	$2/5$	$1/9+1/9=2/9$

直到 x^3 都精确；在 x^4 处出错。

为什么是 $2n-1$ 次。 有 n 个点的法则有 $2n$ 个数可选：n 个位置和 n 个权重。对 $1,x,\dots,x^{2n-1}$ 精确是 $2n$ 个条件。高斯证明了这些条件总能满足，而且所有位置都在区间内，所有权重都为正。

练习 34.6

思路。 光滑函数的最小值处导数为零。

正规方程。 $\|y-X\beta\|^2=(y-X\beta)^{\top}(y-X\beta)$。它对 β 的导数是 $-2X^{\top}(y-X\beta)$。令它为零，得到 $X^{\top}X\beta=X^{\top}y$。

直线。 由 $\bar{x}=1$ 和 $\bar{y}=5/3$，斜率为

$$b=\frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sum(x_i-\bar{x})^2}=\frac{(-1)(-\frac{2}{3})+0+(1)(\frac{1}{3})}{2}=0.5,$$

而 $a=\bar{y}-b\bar{x}=5/3-1/2=7/6\approx 1.1667$。拟合值 $7/6,5/3,13/6$ 留下误差 $-1/6,1/3,-1/6$，它们加起来为零，且与 $(0,1,2)$ 垂直：正规方程得到验证。

练习 34.7

带保护的隐含波动率求解器，与纯牛顿法对比。

```
import numpy as np


def bs_call(s, k, t, vol):
    d1 = (math.log(s / k) + (r + 0.5 * vol**2) * t) / (vol * math.sqrt(t))
    return s * N(d1) - k * math.exp(-r * t) * N(d1 - vol * math.sqrt(t))


def bs_vega(s, k, t, vol):
    d1 = (math.log(s / k) + (r + 0.5 * vol**2) * t) / (vol * math.sqrt(t))
    return s * math.exp(-d1 * d1 / 2) / math.sqrt(2 * math.pi) * math.sqrt(t)


def implied_vol(price, k, t, lo=1e-4, hi=5.0, tol=1e-12):
    # Newton from the middle of a bracket; any step that leaves the bracket becomes a
bisection step.
    f = lambda v: bs_call(S0, k, t, v) - price
    flo = f(lo)
    x = 0.3
    for _ in range(100):
        fx = f(x)
        if abs(fx) < tol:
            return x
        if (fx > 0) == (flo > 0):          # keep the bracket [lo, hi] around the root
            lo, flo = x, fx
        else:
            hi = x
        vg = bs_vega(S0, k, t, x)
        step = x - fx / vg if vg > 1e-12 else None
        x = step if step is not None and lo < step < hi else 0.5 * (lo + hi)
    return x


def plain_newton(price, k, t, x=0.2):
    for _ in range(50):
        vg = bs_vega(S0, k, t, x)
        if not (vg > 1e-12) or not math.isfinite(x) or x <= 0 or x > 50:
            return None
        x = x - (bs_call(S0, k, t, x) - price) / vg
    return x


rng = np.random.default_rng(347)
worst, fails, cases, skipped = 0.0, 0, 0, 0
for k in np.linspace(50, 200, 20):
    for t in np.geomspace(0.1, 5, 10):
        v_true = float(rng.uniform(0.10, 0.60))
        price = bs_call(S0, float(k), float(t), v_true)
        if price - max(S0 - k * math.exp(-r * t), 0) < 1e-6:    # no time value: the price
says nothing about sigma
            skipped += 1
            continue
        cases += 1
```

```
        worst = max(worst, abs(implied_vol(price, float(k), float(t)) - v_true))
        pn = plain_newton(price, float(k), float(t))
        fails += pn is None or abs(pn - v_true) > 1e-6
print(f"cases {cases} (skipped {skipped} with time value below 1e-6), largest error of the
safeguarded solver {worst:.1e}")
print(f"plain Newton from 20% failed in {fails} cases")
```

它的输出。

```
cases 187 (skipped 13 with time value below 1e-6), largest error of the safeguarded solver
6.6e-10
plain Newton from 20% failed in 51 cases
```

读输出。 200 种情形中有 13 种被跳过：这些期权深度实值、期限极短，它们的价格在 10^{-6} 以内就等于内在价值。它们的价格不含关于 σ 的任何信息，没有哪个求解器能把它反推出来。在其余 187 种情形上，带保护的求解器精确到 6.6e-10。从 20% 出发的纯牛顿法在其中 51 种情形上失败：凡是 20% 处 vega 很小的地方，它的第一步就会冲过头，再也找不回来。

练习 34.8

从三个起点出发的 Levenberg-Marquardt，有约束和无约束，然后在 τ 上做网格搜索。

```
tm = np.array(o["fit"]["t"])
yld = np.array(o["fit"]["y_pct"]) / 100
g = lambda t, tau: (1 - np.exp(-t / tau)) / (t / tau)
model = lambda p: p[0] + p[1] * g(tm, p[2])


def jac(p):
    x = tm / p[2]
    dg = (np.exp(-x) * (1 + x) - 1) / x**2
    return np.column_stack([np.ones_like(tm), g(tm, p[2]), p[1] * dg * (-tm / p[2] ** 2)])


def lm(p0, tau_min=None, iters=60):
    p = np.array(p0, float); lam = 1e-2
    for _ in range(iters):
        res = yld - model(p); cur = float(res @ res)
        J = jac(p); A = J.T @ J
        if np.linalg.matrix_rank(A) < 3:
            return p, cur, "the Jacobian has a zero column: no information about tau"
        while lam < 1e12:
            pn = p + np.linalg.solve(A + lam * np.diag(np.diag(A)), J.T @ res)
            if tau_min is not None:
                pn[2] = max(pn[2], tau_min)
            if pn[2] > 0 and float((yld - model(pn)) @ (yld - model(pn))) < cur:
                p, lam = pn, lam / 3
                break
            lam *= 3
        else:
            break
    res = yld - model(p)
    return p, float(res @ res), "ok"


for bound in (None, 0.05):
    print("tau >= 0.05" if bound else "no bound")
    for start in ((0.05, -0.03, 20.0), (0.03, 0.01, 0.5), (0.04, 0.0, 2.0)):
        p, sse, note = lm(start, bound)
        print(f"  start {start}:  b0 {p[0]:.5f}  b1 {p[1]:+.5f}  tau {p[2]:8.4f}  SSE
{sse:.2e}  {note}")

# What does work: for each tau on a grid, b0 and b1 enter linearly, so solve for them by
least squares;
# keep the best tau, then polish all three with Levenberg-Marquardt.
best = None
for tau in np.geomspace(0.1, 30, 60):
    X = np.column_stack([np.ones_like(tm), g(tm, tau)])
    beta = np.linalg.lstsq(X, yld, rcond=None)[0]
    res = yld - X @ beta
    if best is None or res @ res < best[0]:
        best = (float(res @ res), beta[0], beta[1], tau)
p, sse, note = lm(best[1:])
```

```
print(f"grid over tau, then LM: start tau {best[3]:.3f} -> b0 {p[0]:.5f}  b1 {p[1]:+.5f}  tau
{p[2]:.4f}  SSE {sse:.2e}")
```

它的输出。

```
no bound
  start (0.05, -0.03, 20.0):  b0 0.04200  b1 -0.01389  tau   2.0280  SSE 4.64e-08  ok
  start (0.03, 0.01, 0.5):  b0 0.03612  b1 +0.00006  tau   0.1036  SSE 9.50e-05  the Jacobian
has a zero column: no information about tau
  start (0.04, 0.0, 2.0):  b0 0.04000  b1 +0.00000  tau   2.0000  SSE 1.70e-04  the Jacobian
has a zero column: no information about tau
tau >= 0.05
  start (0.05, -0.03, 20.0):  b0 0.04200  b1 -0.01389  tau   2.0280  SSE 4.64e-08  ok
  start (0.03, 0.01, 0.5):  b0 0.04064  b1 -0.00056  tau   0.0500  SSE 2.05e-04  the Jacobian
has a zero column: no information about tau
  start (0.04, 0.0, 2.0):  b0 0.04000  b1 +0.00000  tau   2.0000  SSE 1.70e-04  the Jacobian
has a zero column: no information about tau
grid over tau, then LM: start tau 2.002 -> b0 0.04200  b1 -0.01389  tau 2.0280  SSE 4.64e-08
```

读输出。 第一个起点成功了。另外两个出于同一个原因失败：它们最终都有 $b_1 \approx 0$。当 $b_1 = 0$ 时，模型就是水平直线 b_0，τ 对它毫无影响：雅可比矩阵中 τ 那一列为零，没有哪一步能说出该往哪个方向移动它。第三个起点从那里开始；第二个漂移到了那里。约束 $\tau \geq 0.05$ 使 τ 保持为正，但没有消除病因。

管用的做法。 对每个固定的 τ，模型对 b_0 和 b_1 是线性的，所以可以用最小二乘精确求出它们。在一个网格上扫描 τ，保留最好的，并从那里开始 Levenberg-Marquardt：它得到正确的答案，网格上的 τ 接近 2.00，精修后误差为 4.6e-08。把线性进入的参数分离出来，是使非线性拟合稳健的标准做法。

自测

1. 牛顿法利用斜率（vega）直接跳到切线与零的交点，所以它的误差每步平方一次；二分法只是把区间减半。牛顿法需要导数和一个好的起点；二分法只需要一个区间。

2. 因为远期利率是 $z + t\,z'$：它取决于零息曲线的斜率，并被 t 放大。零息曲线上的小折角或小波动会变成远期利率上的跳跃和驼峰，而为未来现金流定价的正是远期利率。

3. λ 增大时，步子变短，并直指下坡方向。每当一步降低了误差，λ 就除以 3，使方法向 Gauss-Newton 靠拢；每当一步没有降低误差，λ 就乘以 3。

第 35 课解答

回归与偏微分方程进阶

练习 35.1 至 35.8 以及第 35 课自测题的解答。

练习 35.1

思路。 把误差拆成任何 X 的函数都消除不了的部分，加上 g 能控制的部分。

第 1 步。 写 $m(X) = E[Y \mid X]$。那么

$$\big(Y - g(X)\big)^2 = \big(Y - m(X)\big)^2 + 2\big(Y - m(X)\big)\big(m(X) - g(X)\big) + \big(m(X) - g(X)\big)^2.$$

第 2 步：交叉项的均值为零。 以 X 为条件：因子 $m(X) - g(X)$ 是已知的，可以提出来（第 8 课），剩下 $E[Y - m(X) \mid X] = m(X) - m(X) = 0$。由塔性质，它的平均值也为零。

第 3 步。 所以 $E\big[(Y - g(X))^2\big] = E\big[(Y - m(X))^2\big] + E\big[(m(X) - g(X))^2\big]$。第一项不依赖于 g，第二项永不为负，且当 $g = m$ 时为零。■

练习 35.2

思路。 回归只用来决定是否行权，而这个问题只在实值时才出现。

为什么只用实值路径。 虚值时行权一文不值，所以在那里继续持有价值从不与任何东西比较。它的估计只需在看跌期权处于实值的地方准确。

用所有路径会出什么问题。 少数几个基函数必须描述整个股价范围上的继续持有价值。在行权价以上，继续持有价值趋于平坦，接近零；在行权价以下，它陡峭上升。低次多项式无法同时追随两者，最小二乘拟合只能折中：它弯过去追随平坦区域，在实值区反而更差，而那里恰恰是决定行权的地方。在本课的检验中，直线在实值区的误差，用所有路径拟合时为 2.68，只在实值区拟合时为 0.75。继续持有价值越差，行权规则就越差，价格也就越低。

练习 35.3

思路。加权平均，权重来自核。

第 1 步：权重。取 $h=10$：在 $X=90$ 和 110 处，$K_h = e^{-\frac{1}{2}(10/10)^2} = e^{-1/2} = 0.6065$；在 $X=100$ 处，$K_h = 1$。

第 2 步：估计。

$$\hat{m}(100) = \frac{0.6065\times 10 + 1\times 0 + 0.6065\times 2}{0.6065 + 1 + 0.6065} = \frac{7.2784}{2.2131} = 3.2888.$$

极限。当 $h\to 0$ 时，100 处的权重占主导，$\hat{m}(100)\to 0$：估计就是那一个点自己的 Y，全是噪声。当 $h\to\infty$ 时，所有权重都趋于 1，$\hat{m}(100)\to (10+0+2)/3 = 4$：总平均，完全忽略了 X。

选择 h。两个极端都估计不了条件期望。带宽必须大到能把噪声平均掉，又要小到它所平均的那些点有相近的条件期望。

练习 35.4

思路。对密度的对数求导，比较两边。

第 1 步。写 $z = x - x_0 - at$，于是 $p = (2\pi b^2 t)^{-1/2} e^{-z^2/(2b^2 t)}$。注意 $\partial z/\partial x = 1$，$\partial z/\partial t = -a$。

第 2 步：空间导数。

$$p_x = -\frac{z}{b^2 t}\,p, \qquad p_{xx} = \left(\frac{z^2}{b^4 t^2} - \frac{1}{b^2 t}\right)p.$$

第 3 步：时间导数。把 $\ln p = -\tfrac{1}{2}\ln(2\pi b^2 t) - z^2/(2b^2 t)$ 对 t 求导：

$$\frac{p_t}{p} = -\frac{1}{2t} + \frac{az}{b^2 t} + \frac{z^2}{2b^2 t^2}.$$

第 4 步：右边。

$$\frac{-a\,p_x + \tfrac{1}{2}b^2 p_{xx}}{p} = \frac{az}{b^2 t} + \frac{z^2}{2b^2 t^2} - \frac{1}{2t}.$$

两者一致。$\surd$

练习 35.5

思路。 对前向方程积分，并通过分部积分把导数移过去。在远处，p 及其导数为零，所以所有边界项也为零。

总概率。

$$\frac{d}{dt}\int p\,dx = \int \Big(-(ap)_x + \tfrac{1}{2}(b^2 p)_{xx}\Big)dx = \Big[-ap + \tfrac{1}{2}(b^2 p)_x\Big]_{-\infty}^{\infty} = 0.$$

配对。

$$\begin{aligned}\frac{d}{dt}\int g\,p\,dx &= \int g_t\,p\,dx + \int g\Big(-(ap)_x + \tfrac{1}{2}(b^2 p)_{xx}\Big)dx \\ &= \int \Big(g_t + a\,g_x + \tfrac{1}{2}b^2 g_{xx}\Big)p\,dx = 0,\end{aligned}$$

对第一项把一个导数移过去，对第二项移两个。

为什么这使两者一致。 在 $t=0$ 时，p 是今天的值 x_0 处的尖峰，所以 $\int g\,p\,dx = g(0,x_0)$：向后方程的答案。在 $t=T$ 时，$g(T,x)=h(x)$，所以 $\int g\,p\,dx = \int h(x)\,p(T,x)\,dx$：前向方程的答案。这个量在中间不变，所以两者相等。

练习 35.6

思路。 和第 29 课一样，只是换成两个变量。

第 1 步：导数。 取 $V(t,S_1,S_2)=U(\tau,x,y)$：$S_1V_1 = U_x$，$S_1^2V_{11} = U_{xx}-U_x$，$S_2V_2 = U_y$，$S_2^2V_{22} = U_{yy}-U_y$，以及 $S_1S_2V_{12} = U_{xy}$（把 U_x/S_1 对 S_2 求导）。而 $V_t = -U_\tau$。

第 2 步：代入。

$$-U_\tau + \tfrac{1}{2}\sigma_1^2(U_{xx}-U_x) + \tfrac{1}{2}\sigma_2^2(U_{yy}-U_y) + \rho\sigma_1\sigma_2 U_{xy} + rU_x + rU_y = rU.$$

合并 U_x 和 U_y 项，就得到本课的方程，其中 $\nu_i = r - \tfrac{1}{2}\sigma_i^2$。系数都是常数，和一维时一样。

每步的工作量，取 $M=100$：

- **完全隐式：** 带宽为 M 的带状方程组中有 M^2 个未知量。它的分解约需 $M^2\times M^2 = M^4 = 100{,}000{,}000$ 次运算。

- **ADI：** 两次扫描，每次是 M 个规模为 M 的三对角方程组，每个约需 $8M$ 次运算：大约 $16M^2 = 160{,}000$ 次。

工作量约少 600 倍，而且网格越细，差距越大。

练习 35.7

用多项式回归和核回归的 Longstaff-Schwartz 方法。

```
import numpy as np

dates, paths = 50, 50_000
dt = T / dates
df = math.exp(-r * dt)
rng = np.random.default_rng(357)
S = S0 * np.exp(((r - 0.5 * sigma**2) * dt + sigma * math.sqrt(dt) *
rng.standard_normal((paths, dates))).cumsum(axis=1))


def lsm(regress):
    cash = np.maximum(K - S[:, -1], 0.0)
    for i in range(dates - 2, -1, -1):
        cash *= df
        itm = K - S[:, i] > 0
        cont = regress(S[itm, i], cash[itm])
        ex = K - S[itm, i]
        do = ex > cont
        cash[np.where(itm)[0][do]] = ex[do]
    return float((cash * df).mean())


def poly3(x, y):
    return np.polyval(np.polyfit(x / K, y, 3), x / K)


def kernel(h):
    def fit(x, y):
        grid = np.linspace(x.min(), x.max(), 120)              # estimate on a grid, then
interpolate
        wts = np.exp(-0.5 * ((grid[:, None] - x[None, :]) / h) ** 2)
        m = (wts @ y) / wts.sum(axis=1)
        return np.interp(x, grid, m)
    return fit


print("regression            American put")
res = {"poly": lsm(poly3)}
print(f"cubic polynomial      {res['poly']:.4f}")
for h in (0.5, 1.0, 2.0, 5.0, 15.0):
    res[h] = lsm(kernel(h))
    print(f"kernel, h = {h:4.1f}      {res[h]:.4f}")
print("tree (Lesson 22): about 9.478")
```

它的输出。

```
regression            American put
cubic polynomial      9.4540
kernel, h =  0.5      9.5209
kernel, h =  1.0      9.4600
kernel, h =  2.0      9.2886
kernel, h =  5.0      8.6914
kernel, h = 15.0      8.5087
tree (Lesson 22): about 9.478
```

读输出。 答案强烈依赖于带宽：从 $h = 0.5$ 时的 9.5209 到 $h = 15$ 时的 8.5087。小带宽拟合的是每条路径自己的未来现金流：规则在同一批路径上拟合和估值，于是获得了一点先见之明，价格向上偏，这里高于树方法的 9.478（第 30 课，练习 30.6）。大带宽把继续持有价值压平，使行权边界模糊：规则很差，价格向下偏。在 $h = 1$ 时为 9.4600，接近三次多项式的 9.4540。核回归需要谨慎选择带宽，最好是在新的路径上估值；用少数几个多项式项则远没有这么挑剔。

练习 35.8

用 Craig-Sneyd 为篮子看涨期权定价，并用蒙特卡罗检验。

```
a1, a2, s1, s2, rho = 100.0, 95.0, 0.20, 0.30, 0.5


def thomas(a, b, c, d):
    n = len(b); cp = np.zeros(n); dp = np.zeros_like(d, dtype=float)
    cp[0] = c[0] / b[0]; dp[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / m if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / m
    x = np.zeros_like(dp); x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


def craig_sneyd(h, payoff, edge_value, half=1.5, damp=4):
    n_half = math.ceil(half / h)
    x = math.log(a1) + h * np.arange(-n_half, n_half + 1)
    y = math.log(a2) + h * np.arange(-n_half, n_half + 1)
    n = len(x); N_ = n - 1; k = T / N_
    nu1, nu2 = r - 0.5 * s1**2, r - 0.5 * s2**2
    X, Y = np.meshgrid(np.exp(x), np.exp(y), indexing="ij")
    U = payoff(X, Y)
    A1 = lambda V: np.pad(nu1 * (V[2:] - V[:-2]) / (2 * h) + 0.5 * s1**2 * (V[2:] - 2 *
V[1:-1] + V[:-2]) / h**2 - 0.5 * r * V[1:-1], ((1, 1), (0, 0)))
    A2 = lambda V: np.pad(nu2 * (V[:, 2:] - V[:, :-2]) / (2 * h) + 0.5 * s2**2 * (V[:, 2:] -
2 * V[:, 1:-1] + V[:, :-2]) / h**2 - 0.5 * r * V[:, 1:-1], ((0, 0), (1, 1)))
    A0 = lambda V: np.pad(rho * s1 * s2 * (V[2:, 2:] - V[2:, :-2] - V[:-2, 2:] + V[:-2, :-2])
/ (4 * h * h), ((1, 1), (1, 1)))

    def solve(nu_, s_, rhs, th):
        lo, mi, up = 0.5 * s_**2 / h**2 - nu_ / (2 * h), -s_**2 / h**2 - 0.5 * r, 0.5 * s_**2
/ h**2 + nu_ / (2 * h)
        a = np.full(n, -th * k * lo); b = np.full(n, 1 - th * k * mi); c = np.full(n, -th * k
* up)
        a[0] = c[0] = a[-1] = c[-1] = 0.0; b[0] = b[-1] = 1.0
        return thomas(a, b, c, rhs)

    def sweeps(Y0, a1U, a2U, th, edge):
        Y1 = solve(nu1, s1, Y0 - th * k * a1U, th); Y1[0], Y1[-1] = edge[0], edge[-1]
        Y2 = solve(nu2, s2, (Y1 - th * k * a2U).T, th).T
        Y2[:, 0], Y2[:, -1], Y2[0], Y2[-1] = edge[:, 0], edge[:, -1], edge[0], edge[-1]
        return Y2

    for step in range(N_):
        th = 1.0 if step < damp else 0.5
        edge = edge_value(X, Y, (step + 1) * k)
        a0U, a1U, a2U = A0(U), A1(U), A2(U)
        Y0 = U + k * (a0U + a1U + a2U)
        U2 = sweeps(Y0, a1U, a2U, th, edge)
        U = sweeps(Y0 + 0.5 * k * (A0(U2) - a0U), a1U, a2U, th, edge)
    return float(U[n_half, n_half]), n
```

```
basket = lambda X, Y: np.maximum(0.5 * (X + Y) - 100, 0)
basket_edge = lambda X, Y, tau: np.maximum(0.5 * (X + Y) - 100 * math.exp(-r * tau), 0)   #
deep in or out of the money
for h in (0.05, 0.025, 0.0125):
    v, n = craig_sneyd(h, basket, basket_edge)
    print(f"Craig-Sneyd, h = {h:.4f} ({n} x {n} grid): {v:.4f}")

rng = np.random.default_rng(358)
Z = rng.standard_normal((2_000_000, 2))
Z2 = rho * Z[:, 0] + math.sqrt(1 - rho**2) * Z[:, 1]                # Lesson 31's Cholesky, 2
x 2
ST1 = a1 * np.exp((r - 0.5 * s1**2) * T + s1 * math.sqrt(T) * Z[:, 0])
ST2 = a2 * np.exp((r - 0.5 * s2**2) * T + s2 * math.sqrt(T) * Z2)
pay = math.exp(-r * T) * np.maximum(0.5 * (ST1 + ST2) - 100, 0)
mc, se = float(pay.mean()), float(pay.std() / math.sqrt(len(pay)))
print(f"Monte Carlo, 2,000,000 paths: {mc:.4f} (standard error {se:.4f})")
```

它的输出。

```
Craig-Sneyd, h = 0.0500 (61 x 61 grid): 8.6045
Craig-Sneyd, h = 0.0250 (121 x 121 grid): 8.6136
Craig-Sneyd, h = 0.0125 (241 x 241 grid): 8.6157
Monte Carlo, 2,000,000 paths: 8.6110 (standard error 0.0101)
```

读输出。 网格价格稳定在 8.6157，模拟给出 8.6110，标准误差为 0.0101：两者在一个标准误差之内吻合。代码中有两个细节：边界值用贴现后的行权价 $\max\left(\frac{1}{2}(S_1+S_2)-100e^{-r\tau},0\right)$，这在深度实值或深度虚值时都是正确的值；收益的拐点，即直线 $S_1+S_2=200$，不落在网格点上，这使收敛不如 Margrabe 那样规则，但仍然稳定。

自测

1. 在给定今天股价时贴现收益的条件期望：继续持有价值，在所用的那一族基函数内估计。

2. 在时间上向前，从今天开始。它把股价（或其对数）的密度从今天的尖峰推进到以后时刻铺开的形状。

3. 每次隐式修正只沿一个方向联系网格点，所以它是一批三对角方程组，总共约需 M^2 次运算，和显式步一样。Craig-Sneyd 增加一个阶段，用新值重新计算交叉项，再重复两次修正，从而把时间上的精度从一阶提高到二阶。